How Docker works

What a container is, what `docker run` does, and how ports and volumes fit in.

The one-minute version

- An **image** is a packaged program: the app plus everything it needs to run (system files, Python, libraries), frozen into read-only **layers**.
- A **container** is a running copy of an image. It is an ordinary process on the server, fenced off so that it sees only its own files, processes and network.
- Containers are **not virtual machines**. They share the server's Linux kernel, so they start in a second and cost almost nothing extra.
- A container is disposable: what it writes is lost when you delete it, unless you keep it in a **volume**. To reach it from outside, you **publish a port**.

---

1 Why Docker exists

"It works on my machine." A program needs the right version of Python, the right libraries, the right system tools and settings. Your laptop has one mix, the server another, a classmate's laptop a third. Installing everything by hand on every machine is slow, and two programs often need versions that conflict.

Docker packs a program *together with its whole environment* into one image. The same image runs the same way on your laptop, on the class server and in the cloud. You do not install Open WebUI, its Python and its hundreds of libraries: you run one image, and delete it when you are done, leaving nothing behind.

This is how most software is deployed today. Later in this course, **vLLM** runs from a Docker image, and **Modal** runs your training inside containers built from images.

---

2 Images and containers

Image: `nginx:1.27-alpine`

- The packaged program. **Read-only**: it never changes.
- Downloaded with `docker pull`, or built with `docker build`.
- Listed by `docker images`.

Container: `web`

- One running (or stopped) copy of an image, with a name you choose.
- Made with `docker run`. One image can have many containers.
- Listed by `docker ps` (running) and `docker ps -a` (all).

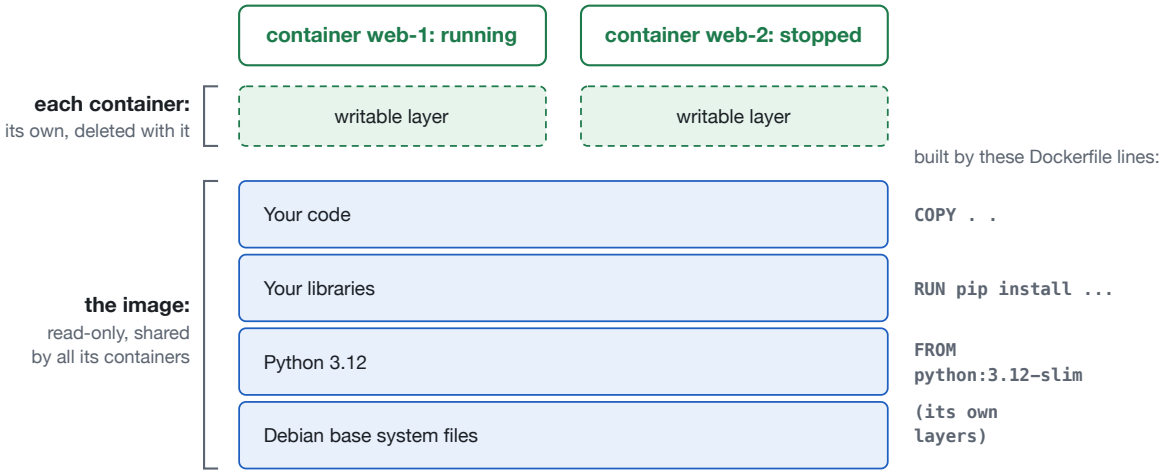
An analogy you know from Python: an image is a **class**, a container is an **object**. You define the class once; you can make many objects from it; each object has its own state; deleting an object does not touch the class.

Image names

`ghcr.io/open-webui/open-webui:main-slim` reads as *registry* `ghcr.io` (GitHub's), *name* `open-webui/open-webui`, *tag* `main-slim` (a version or variant). With no registry, Docker uses **Docker Hub**: `nginx` means `docker.io/library/nginx`. With no tag, it means `:latest`, so always give a tag when you care which version you get.

Layers

An image is a stack of read-only **layers**, one for each step that built it. A running container adds one thin **writable layer** on top. When the container changes a file, the changed copy goes into its writable layer; the image below stays untouched. That is why many containers can share one image, and why deleting a container throws away exactly what it wrote.

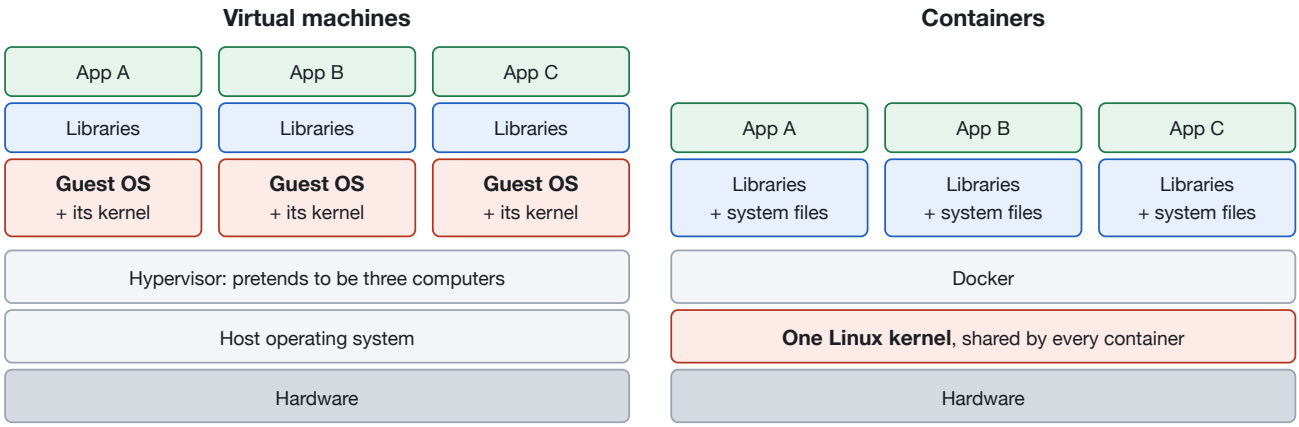


Two containers from one image. The image's layers are shared and never change; each container writes only into its own top layer.

---

3 Containers are not virtual machines

A **virtual machine** pretends to be a whole computer and runs a complete operating system of its own, kernel included. A **container** is just a process on the server's own Linux, run with fences around it.



Red marks a kernel. Three virtual machines run three operating systems; three containers share one kernel.

	Virtual machine	Container
Starts in	tens of seconds: it boots an operating system	about a second: it starts one process
Size	gigabytes: a whole operating system	megabytes to a few gigabytes: only what the app needs
Can run	any operating system, Windows included	Linux programs, on the host's Linux kernel
Isolation	strong: separate kernels	good, but one shared kernel

Your class server is itself a virtual machine in Google's data centre, and your containers run inside it.

The fences: namespaces and cgroups

Two features of the Linux kernel make a process into a container:

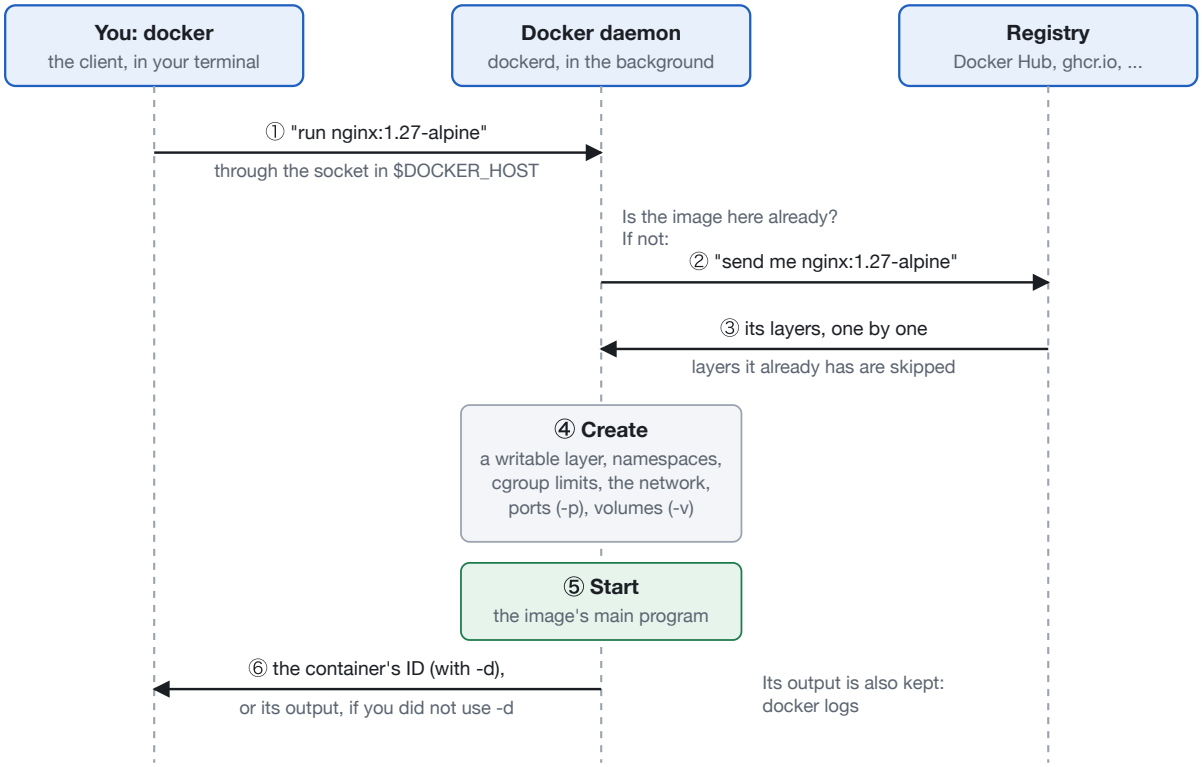
- **Namespaces** decide what a process can **see**. In its own namespaces, a container sees its own files (the image's layers), its own list of processes, its own network and its own hostname. Its main program believes it is process number 1.
- **Control groups (cgroups)** decide how much a process can **use**: memory, CPU time, number of processes. `docker run -m 100m` caps a container at 100 MB. The same mechanism caps *you* at 3 GB and 2 CPUs on the class server.

You can see both in section 12.

---

4 What happens when you type `docker run`

The `docker` command does not run containers itself. It is a **client** that sends your request to the **Docker daemon** (`dockerd`), a program that runs in the background and does the work. The variable `$DOCKER_HOST` tells the client where to find the daemon.



One `docker run`, step by step. Dashed lines are time moving down; each arrow is a message.

1. **The client asks the daemon.** `docker run` sends the request over a **socket**, a file-like connection named in `$DOCKER_HOST`. If no daemon answers, you get *Cannot connect to the Docker daemon*.
2. **Find the image.** If the image is not on the machine yet, the daemon downloads it from a **registry**, layer by layer, skipping layers it already has. This happens once; the next `docker run` of the same image starts at once.
3. **Create the container.** A writable layer on top of the image, its own namespaces, cgroup limits, a private network, and anything you asked for: published ports (`-p`), volumes (`-v`), settings (`-e`), a name (`--name`).
4. **Start the main program.** Each image names one main program (its `CMD`). **The container lives exactly as long as that program does.** When it finishes or crashes, the container stops, shown as `Exited (code)` in `docker ps -a`. Exit code 0 means it finished normally; anything else, look at `docker logs`.

5 **Detached or attached.** With `-d` the container runs in the background and you get your prompt back. Without it, your terminal shows the program's output until it stops. Either way, `docker logs NAME` shows everything it printed.

`docker stop` ends the main program; the container stays, stopped, and `docker start` runs it again with its writable layer intact. `docker rm` deletes the container and its writable layer. `docker run --rm` deletes it by itself when it stops.

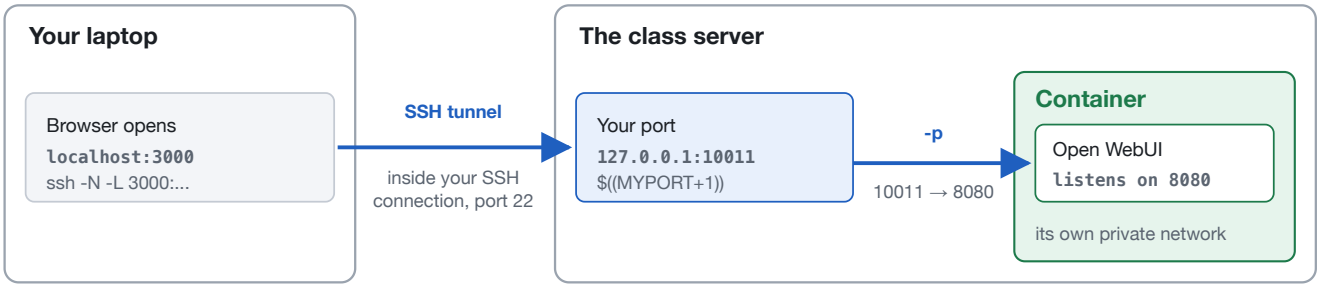
---

5 Ports: reaching a container

A container has its own private network. A program inside that listens on port 8080 is listening on the *container's* port 8080, which nothing outside can reach. **Publishing** a port connects a port on the server to a port in the container:

```
docker run -d -p 127.0.0.1:10011:8080 ...
```

The **blue part** is on the server, the **green part** inside the container: read it as **outside : inside**. `127.0.0.1` means the server accepts connections only from itself, so the port is not open to the internet. Leave it out and the port opens on every network address the server has.



Mission 3's request path. The tunnel brings your browser to the server; the published port brings it into the container.

- **One port, one program.** Two containers cannot publish the same server port. On the class server everyone shares one machine, so each of you has ten ports of your own, `$MYPORT` to `$MYPORT+9`. Inside containers, ports never clash: every container has its own 8080.
- **The two numbers must match reality.** The inside number must be the port the program actually listens on. Publish the wrong one and the container runs, but nothing answers. `docker port NAME` shows what is published; the program's own log usually says where it listens.
- `EXPOSE 8080` in a Dockerfile is only a note for humans. It publishes nothing; `-p` does.

---

6 Keeping data: volumes and bind mounts

Treat containers as disposable: you delete them and make new ones all the time, to upgrade an image or change a setting. Whatever a container wrote into its writable layer goes with it. Data that must survive goes somewhere else, **mounted** into the container at a path you choose:

	Named volume	Bind mount
Looks like	`-v openwebui-data:/app/backend/data`	`-v ~/s2/site:/usr/share/nginx/html`
What it is	Storage that Docker manages, under a name	A folder of yours, shown inside the container
Use it for	An app's own data: databases, accounts, chats	Files you edit yourself: code, a web page, config

	Named volume	Bind mount
Lives until	`docker volume rm`; deleting containers leaves it	You delete the folder
See it with	`docker volume ls`	`ls`, on the server

The rule for `-v`: if the left side starts with `/` or `~` (or `./`), it is a folder; otherwise it is a volume name. The right side is always the path inside the container. Both are the same file, not a copy: change it on one side and the other sees it at once.

Settings go in as environment variables: `-e TZ=Asia/Yerevan`. **Secrets** such as API keys never go into an image, a Dockerfile or a `compose.yml`: those get shared and committed to git. Type them into the app, or keep them in a `.env` file that git ignores.

---

7 Building your own image: the Dockerfile

A **Dockerfile** is the recipe for an image: a starting image, then steps. Each step adds a layer.

```
FROM python:3.12-slim                  # start from an existing image
WORKDIR /app                           # the folder later steps work in
COPY requirements.txt .                # copy a file from your folder into the image
RUN pip install -r requirements.txt    # run a command while building
COPY . .                               # copy the rest of your code
ENV PORT=8000                          # a default setting
EXPOSE 8000                            # a note: the app listens here
CMD ["python", "main.py"]              # the main program, run when a container starts
```

```
docker build -t myapp .                # build an image called myapp from the Dockerfile here
docker run -d --name app -p 127.0.0.1:$MYPORT:8000 myapp
```

- **Build time and run time.** `RUN` happens once, while building. `CMD` happens every time a container starts. A mistake in the first shows up in `docker build`; a mistake in the second shows up in `docker logs`.
- **The dot** at the end of `docker build` is the **build context**: the folder whose files `COPY` can see. `COPY` can only copy files that exist there, under exactly that name.
- **The cache.** Docker reuses every layer up to the first step that changed. That is why the slow `pip install` comes before `COPY . .`: editing your code then rebuilds in a second.
- **Changing code means rebuilding.** A container runs the image it was made from. After editing code: `docker build` again, remove the old container, run a new one.

---

8 Compose: the whole setup in one file

A long `docker run` command is easy to mistype and hard to remember. **Docker Compose** writes the same thing down in `compose.yml`, so that anyone can start it with one command. An example:

```
services:
  web:                                 # the service's name
    image: nginx:1.27-alpine
    ports:
      - "127.0.0.1:${PORT}:80"         # PORT comes from the file .env
    volumes:
      - ./site:/usr/share/nginx/html   # a bind mount
    restart: unless-stopped            # come back after a crash or a restart
```

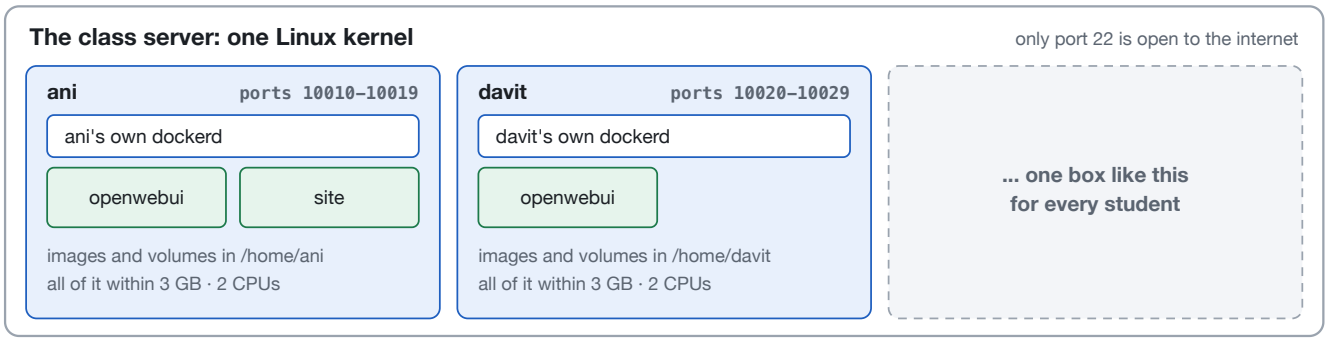
Command (in the folder with `compose.yml`)	What it does
`docker compose up -d`	Create and start everything, in the background. Run it again after editing the file: it changes only what changed.
`docker compose ps` / `logs -f`	Status, and output, of this file's containers
`docker compose down`	Stop and delete the containers. Named volumes stay, unless you add `-v`.

Restart policies say what happens when the main program stops or Docker itself restarts: `no` (the default: stay stopped), `on-failure` (restart after a crash), `always`, and `unless-stopped` (always, unless you stopped it yourself). A service that should just be there, like your Open WebUI, uses `unless-stopped`.

---

9 Docker on our class server

On most servers there is one Docker daemon, running as root, shared by everyone. That is unsafe on a shared machine: **whoever can use that daemon can become root**, for example by mounting the server's whole disk into a container. So the class server gives each of you **your own daemon, running as you**. This is called **rootless Docker**.



The ports and names are examples. `echo $MYPORT` shows your first port.

- `docker ps` shows **only your** containers and `docker images` only your images. Two students can both have a container called `openwebui`.
- You need no `sudo`. Root *inside* your containers is only your own user outside them.
- Your containers count against **your** limits: 3 GB of memory and 2 CPUs for everything you run.
- `$DOCKER_HOST` points at your daemon's socket, `/run/user/<your id>/docker.sock`. If your daemon is not running: `systemctl --user start docker`.
- **Ports are the one thing you share.** They all belong to the same server, so use only your own ten. `127.0.0.1` hides a port from the internet, but not from other users on the same server: that is why mission 3 has you create your Open WebUI account at once.

---

10 Commands you will use

Command	What it does
`docker run -d --name web IMAGE`	Make and start a container in the background. Add `-p`, `-v`, `-e` as needed
`docker run --rm -it IMAGE sh`	A throwaway container with a shell inside; deleted when you exit
`docker ps` / `docker ps -a`	Running containers / all containers, stopped ones too

Command	What it does
`docker logs -f web`	What the container printed; `-f` keeps following (Ctrl+C stops following, not the container)
`docker exec -it web sh`	A shell inside a running container. `exit` leaves it
`docker stop web` / `docker start web`	Stop the main program / start it again
`docker rm web` / `docker rm -f web`	Delete a stopped container / stop and delete it
`docker port web`	Which server port goes to which container port
`docker images` / `docker pull IMAGE`	Your images / download one
`docker build -t NAME .`	Build an image from the Dockerfile in this folder
`docker volume ls` / `docker volume rm NAME`	Your volumes / delete one, with its data
`docker inspect web`	Everything about a container, as JSON
`docker system df`	How much disk your images, containers and volumes use

11 When it goes wrong

You see	What it means and what to do
`Cannot connect to the Docker daemon`	No daemon is answering. On the class server: `systemctl --user start docker`, wait a few seconds, try again. On a laptop: start Docker Desktop.
`Conflict. The container name "/web" is already in use`	A container with that name exists, perhaps stopped. `docker ps -a` shows it; `docker rm -f web` deletes it, or pick another name.
`port is already allocated` or `address already in use`	Something already uses that server port: one of your own containers (`docker ps`) or someone else's. Use a free port from your own range.
The container shows `Exited (1)` right after starting	Its main program crashed, so the container stopped. `docker logs NAME` shows the program's last words, usually the reason.
`Exited (137)`	The program was killed, most often for using more memory than its limit.
`curl` says `Connection refused`, or the connection is reset	Nothing answers at that address. Is the container running (`docker ps`)? Does `docker port NAME` point to the port the program really listens on? Did you use your own port number?
`pull access denied` or `manifest unknown`	The image's name or tag is misspelled, or that tag does not exist.
Your code change does not show up	The container still runs the old image. Rebuild, remove the container, run it again.
The app starts empty: your data is gone	The new container is not using the old volume. Check the volume's name after `-v` (or in `compose.yml`) and `docker volume ls`.

12 Try it yourself (5 minutes, on the class server)

1. **Another system, the same kernel.** Compare the server with a container:

```
cat /etc/os-release | head -1;  uname -r
docker run --rm python:3.12-slim sh -c 'head -1 /etc/os-release; uname -r'
```

The server runs Ubuntu, the container Debian, yet `uname -r`, the kernel's version, is the same. Which row of the table in section 3 does this show?

2. **A world of its own.** `docker run --rm python:3.12-slim python -c "import os; print(os.getpid(), os.uname().nodename)"`. The program is process number 1, and its hostname is not the server's: namespaces.
3. **Layers.** `docker history nginx:1.27-alpine` lists the steps that built the image, with each layer's size.
4. **A cgroup limit.** Give a container 100 MB and ask for 300 MB:

```
docker run --rm -m 100m python:3.12-slim python -c "x = bytearray(300_000_000)"; echo
"exit code: $?"
```

It is killed, with exit code 137. You are under the same kind of limit, only bigger.

5. **Think:** why is it safe to delete a container, but not a volume?

---

13 Words to know

Image	A packaged program and its environment, in read-only layers.
Container	A running (or stopped) copy of an image: a fenced-off process.
Layer	One step's worth of files in an image. Shared and cached.
Registry	A server that stores images: Docker Hub, `ghcr.io`.
Tag	The version part of an image's name, after the colon.
Dockerfile	The recipe that builds an image.
Build context	The folder `docker build` can copy files from.
Daemon	`dockerd`: the background program that runs containers.
Publish a port	Connect a server port to a container port: `-p outside:inside`.
Volume	Docker-managed storage that outlives containers.
Bind mount	A folder of yours, shown inside a container.
Compose	A container setup written in `compose.yml`, run with one command.
Restart policy	What Docker does when a container's program stops.
Namespace	A kernel fence for what a process can see.
Cgroup	A kernel limit on what a process can use.
Rootless Docker	A Docker daemon that runs as an ordinary user, not root.